

# PRD · Nova Trade-Doc Validation (Part 1)

**Author:** Poonam · **Date:** 2026-05-29 · **Status:** Draft for review **What this is:** A 3-agent pipeline (Extractor · Validator · Router) plus a queryable persistence layer and a minimal UI · built as the *plan + execute* core of Nova's five-stage pipeline. Self-hosted vision and text models served via `vllm serve` against the OpenAI-compatible `/v1/chat/completions` endpoint; orchestration via LangGraph (Nova's actual stack).

---

## Table of Contents

- 1 Problem & Failure Modes
  - 2 First 5 Minutes · the Operator Experience
  - 3 Users & JTBDs
  - 4 Nova Alignment · Why this product *Nova-shaped*, not just multi-agent
  - 5 Agent Architecture
  - 6 LLM, Tooling & Orchestration
  - 7 Trust, Hallucination, Evals
  - 8 Observability
  - 9 Cost & Latency Budget
  - 10 Metrics & Pilot Go/No-Go
  - 11 Demo Plan · A through E on real input
  - 12 What's Next
  - 13 Appendices · schemas, decision tables, repo layout
- 

## 1. Problem

Every international shipment generates 4-10 documents · Bill of Lading, Commercial Invoice, Packing List, Certificate of Origin. They arrive as emailed PDFs and scans. A CG operator opens each one, reads fields by eye, cross-checks against a customer-specific rule set, and decides: accept, ask for an amendment, or escalate. This eats hours per shipment and the worst errors (wrong HS code, mismatched consignee, value off by a digit) are caught at customs · at four to five figures per incident.

### 1.1 Where the current trade-doc flow breaks · named failure modes

These are the failure modes that justify Nova existing. Each is concrete, not "AI could help":

- 1 **Silent field omission on bad scans.** OCR drops the seal number from a smudged BoL. Nobody notices until the container is held at the port.
- 2 **Cross-document value drift.** BoL declares \$12,450; commercial invoice says \$12,400. Within tolerance for one customer, a compliance flag for another. Today, only senior ops catches this.
- 3 **Tacit rules that live in heads.** "ACME's HS codes always need double-check because of the Q3 2025 customs incident." Nowhere in any config screen. Leaves with the senior ops person.
- 4 **Email threads as workflow.** Amendment requests get buried in reply chains. No SLA timer, no audit trail of who agreed to what.
- 5 **Re-validation is impossible after rule changes.** A customer updates their approved HS list. There's no cached extraction to re-evaluate · every historical doc would need re-OCR. So rule changes only apply going forward.
- 6 **Cross-doc reconciliation is manual.** Comparing 6 documents per shipment, field by field, against a contract + customs broker portal + ERP entries.
- 7 **After-hours latency.** Docs arrive at 11pm from an Asian supplier. They sit until 9am local. Half a day of SLA is gone.
- 8 **Junior/senior asymmetry.** No system feature distinguishes high-risk from low-risk shipments. So

either juniors get blocked or seniors do everything.

- 9 **Multi-language doc dropouts.** Chinese or Spanish docs get punted to a small group of bilingual staff and become a backlog.
- 10 **No precision in supplier feedback.** When a doc bounces, the message is "please fix" with a screenshot. The supplier guesses what's wrong and sends a revised version with a different mistake.

A traditional logistics SaaS · even an AI-augmented one · can address #4 and #7 at the edges. It cannot touch #1, #2, #3, #5, #6, #8, #10 because those are not workflow problems; they're *judgment + reading + memory* problems. That gap is exactly what Nova's "System of Outcomes" framing is built for.

## 2. First 5 Minutes

This is the experience target. If we don't deliver this on day one, the pilot fails regardless of model quality.

- **t=0s.** Priya logs in. She sees today's shipments sorted by *exception severity*, not arrival time. Green band on top: "12 shipments auto-approved overnight." Red band below: "4 shipments need you."
- **t=30s.** She clicks the top exception. Split view: the source BoL on the left with a yellow box around the HS code field; the extracted value vs. the conflicting invoice value side-by-side on the right. Below: the rule that fired · "ACME approved HS list v3 · 8471.30 not present."
- **t=90s.** She agrees with the system's reading. One click · "Draft amendment request." Nova drafts the email with the specific field, the source page reference, and the corrected value. She skims, sends.
- **t=3m.** Next exception. Value mismatch within tolerance · but flagged because the supplier has done this three times this month. The system surfaces the *pattern*, not just the field. She marks "acceptable this time"; the override is logged with rationale.
- **t=5m.** Three exceptions cleared. Each would have taken 15 minutes manually. The 12 auto-approvals would have been another 90 minutes. Priya has had a 2-hour head start on her day before anyone else has opened Outlook.

**Anti-success:** PDFs the system extracted but didn't highlight. A bare "confidence: 0.83" with no next step. A high-risk field silently approved.

## 3. Users & JTBDs

### 3.1 Personas

**Priya · Senior Trade Ops Analyst (CG operator)** Global pharma importer. Owns 150-200 inbound shipments/week, supervises 3 juniors. Strengths: pattern recognition across customers, knows where every customs broker bends rules. Frustration: 70% of her day is mechanical cross-checking. Win condition: she becomes the *exception handler and supervisor*, not the doer.

**Liang · Export Coordinator (SU supplier)** Contract manufacturer in Shenzhen, 30-60 shipments/month to 12 importers. Sends docs by email; fixes mistakes when importers flag them · often days late, after goods have shipped. Frustration: never knows if a document is "right" until it bounces; each bounce delays payment 3-10 days. Win condition: pre-flight validation before goods leave.

**Dev · Compliance Officer (tertiary, but important for governance scoring)** Audits when things go wrong. Needs traceable decision records, versioned rule sets, per-shipment evidence chains. Win condition: every Nova decision is reproducible six months later.

### 3.2 JTBD statements · each mapped to A-E behaviors

| # | JTBD | A-E | | 1 | **When** a new shipment's docs arrive, **I want** them parsed into structured fields with per-field confidence, **so that** I can trust or distrust each value independently. | A | | 2 | **When** the system extracts fields, **I want** each one's confidence surfaced explicitly, **so that** I review what matters and skim what doesn't. | A | |

3 | **When** extracted fields are validated against my customer's rules, **I want** match/mismatch/uncertain results with " what we found vs. what we expected," **so that** I can act on the discrepancy in seconds. | B | | 4 | **When** a rule is uncertain (not clearly pass/fail), **I want** it surfaced for human review, never silently approved, **so that** the system fails in the safe direction. | B | | 5 | **When** validation finishes, **I want** the router to choose one of auto-approve / human review / amendment-request with a written rationale, **so that** I understand *why* the system did what it did. | C | | 6 | **When** the router asks for an amendment, **I want** the discrepancies enumerated in the message, **so that** the supplier can fix the right thing on the first reply. | C | | 7 | **When** I want to know operational state ("how many flagged this week?"), **I want** to ask in plain language and get a grounded answer from the store, **so that** I don't need an engineer or a BI tool. | D | | 8 | **When** I'm investigating a single shipment, **I want** the UI to show the extracted fields, confidences, validation results, decision, and reasoning side-by-side with the source doc, **so that** I can audit the whole chain in one view. | E | | 9 | **When** I override the router's decision, **I want** my override captured with rationale, **so that** the system has an audit trail and a feedback signal. | (cross-cutting) |

## 4. Nova Alignment

This product is not a generic multi-agent demo. It is shaped specifically to live inside Nova's architectural model, and the design choices below are downstream of that model. Three Nova concepts shape it:

### 4.1 System of Outcomes · the atomic unit ~~is~~ *work*, not *records*

Primary VC's "AI agents need systems of work, not record" makes the case crisply: in the SaaS era the unit was the *record* (data captured for later analysis); in the AI era the unit is *the work itself* · the reasoning, the intermediate steps, the corrections, the why. End-to-end process data is ~10× more valuable than final-decision data, both for auditing and for the feedback loop that makes agents improve.

What this changes architecturally:

- **Every intermediate artifact is persisted.** Not just the final decision, but the extracted JSON (with bboxes, confidences, model versions), the per-rule validation evidence, the router's rationale string. These are the "work units" that the System of Outcomes is built on.
- **Human overrides are first-class artifacts.** When Priya overrides the router, we capture (input, model decision, human decision, rationale). These become labeled examples that re-grade the system over time.
- **Re-validation is cheap.** A customer rule change re-runs the validator against cached extractions · pennies, not dollars; seconds, not hours.
- **Outcome-based north-star.** We measure *Straight-Through Processing rate with zero downstream defect* (see §10), not adoption or login counts. That metric only makes sense if you've captured outcomes; SaaS systems literally can't compute it.

### 4.2 Nova's 5-stage pipeline · and where our 3 agents live in it

Nova's actual orchestration pattern is a five-stage LangGraph pipeline: *scope resolution* · *context compilation* · *schema routing* · *plan + execute* · *evidence delivery*. Our three agents are the **plan + execute** core. The other four stages are real and they're handled by the orchestrator, not by ad-hoc prompts.

| Nova stage | What it does for trade-docs | Owned by | ---|---|---| | 1 · Scope resolution | Identify: which customer, which document, which doc type (classifier), which workflow | Orchestrator (LangGraph node) | | 2 · Context compilation | Load: customer rule set (versioned YAML), prior shipment history, sibling-doc references for cross-doc rules | Orchestrator (RAG retrieval over a Postgres rules store) | | 3 · Schema routing | Select the typed I/O JSON schema for *this* doc type, bind the right field set to the Extractor | Orchestrator (schema registry lookup) | | 4 · Plan + execute | **Extractor** · **Validator** · **Router** (the three agents) | | 5 · Evidence delivery | Persist artifacts to the work store; surface to UI; emit Kafka events; record audit trail | Orchestrator + storage layer |

Why this matters: a one-prompt design collapses stages 1-5 into one opaque call. A poorly-bounded multi-agent design (5 or 7 agents) shoves the orchestrator's responsibilities into agent code, and you lose the ability to swap a

single stage. The 3-agent split aligns with stage 4 precisely; stages 1, 2, 3, 5 stay deterministic.

### 4.3 Generic engine + FDE customization · whatcode vs. what's config

Pragmatic Engineer's piece on Forward Deployed Engineers describes the FDE model: a small team embeds with one customer, owns end-to-end customization, and ships customer-specific deployments on top of a generic platform. Nova explicitly markets "zero hardcoded business logic · the engine is completely generic". That isn't a slogan; it's an *architectural* commitment that decides what an engineer ships once vs. what an FDE ships per customer.

|       |                             |                                      |             |                |                                                            |
|-------|-----------------------------|--------------------------------------|-------------|----------------|------------------------------------------------------------|
| Layer | Generic engine (built once) | FDE / customer config (per customer) | --- --- --- | Document types | The <i>concept</i> of a doc type + extraction schema       |
|       |                             |                                      |             |                | The schema for <i>this</i> customer's BoL variant          |
|       |                             |                                      |             | Validation     | The rule evaluator and three-valued result format          |
|       |                             |                                      |             |                | The customer's YAML rule set, versioned                    |
|       |                             |                                      |             | Routing        | The decision-table executor + LLM rationale generator      |
|       |                             |                                      |             |                | The customer's risk thresholds and decision matrix         |
|       |                             |                                      |             | Storage        | The work-store schema + NL·SQL agent                       |
|       |                             |                                      |             |                | The customer's data residency, retention, and PII policy   |
|       |                             |                                      |             | UI             | The pipeline-view component                                |
|       |                             |                                      |             |                | The customer's branding + workflow integrations            |
|       |                             |                                      |             | Eval           | The eval harness + golden-set tooling                      |
|       |                             |                                      |             |                | The customer's golden set (signed off by their senior ops) |

A new customer should be onboardable by an FDE in **days, not weeks**, because nothing on the right column requires a code change. This is the bar.

## 5. Agent Architecture

### 5.1 Why three agents · and why not one, and why not five

#### One prompt fails because:

- Three jobs (vision extraction, symbolic rule reasoning, decision-making with operational context) need three model classes. One prompt forces the largest model on every step.
- Failures collapse into "the model was wrong somewhere" · no isolation, no targeted retry, no per-stage eval.
- The output is one opaque blob. Trade docs need a defensible audit trail: which field, from which page, against which rule, with which rationale.
- Rule changes re-OCR every historical document. With separation, you replay cached extractions through new rules · orders of magnitude cheaper.

#### Five agents fails because:

- *Per-field extractors* lose cross-field calibration. The VLM disambiguates the consignee partly from knowing the doc is a BoL and that "shipper" is already populated. Splitting fields kills that signal.
- *Per-rule-type validators* lose cross-rule reasoning. An HS-code check can depend on country of origin from another field. One validator with all rules in context wins.
- *Separate OCR pre-processor* is redundant. Modern VLMs do OCR + understanding in one step.
- *Ruleset compiler* (NL · YAML rules) is real, but a *build-time* tool · it belongs in the rules pipeline, not in the per-document runtime path.

**Three is the boundary that aligns with the work shape: perception (Extractor), judgment (Validator), action (Router).** Any finer split fragments work without separating concerns.

The recent multi-agent orchestration benchmark compared sequential pipeline, parallel fan-out with merge, hierarchical supervisor-worker, and reflexive self-correcting loop. For trade-doc validation · where stages are dependent and the workflow shape is fixed · **sequential pipeline is the right pattern**. Reflexive self-correction shows up *within* the Extractor (the re-extraction ladder), but does not become a top-level architectural pattern.

### 5.2 Planner / Executor / Verifier · honest fit

Classic planner/executor/verifier maps imperfectly here. Our pipeline shape is *fixed* in v0 (extract · validate · route), so there is no top-level planner. A true planner agent shows up in v1 when doc-type discovery and cross-doc reconciliation become dynamic.

- **Extractor = Executor.** Produces the structured artifact.
- **Validator = Verifier.** Checks the artifact against rules.
- **Router = Decider.** Commits to next action · not a planner, but the closest thing.

Pretending we need a planner now would be overengineering. Calling it out honestly is itself a design decision.

## 5.3 Per-agent contracts

### Extractor (A)

||| |---|---| | Job | Convert page images · typed, source-grounded JSON. Surface unreadable fields explicitly. || Input | {doc\_id, doc\_type, pages[], expected\_schema\_id} || Output | Per-field {value, confidence, source: {page, bbox}, quote} + unreadable\_fields[] + extractor\_version || Required fields (per brief) | consignee name, HS code, port of loading, port of discharge, Incoterms, description of goods, gross weight, invoice number · each with confidence || Owns | Vision OCR, field structuring, hallucination guards, source attribution || Does NOT own | Rule evaluation, decisions, retries (orchestrator handles) |

### Validator (B)

||| |---|---| | Job | Evaluate every rule in the customer ruleset against extracted JSON. Three-valued results. || Input | {extracted, rule\_set\_id, cross\_doc\_context?} || Output | results[] · each {rule\_id, status: match\|mismatch\|uncertain, confidence, found, expected, reason} || Owns | Rule semantics, cross-field reasoning, evidence trails || Does NOT own | Deciding what to do about mismatches |

Note: vocabulary is **match** / **mismatch** / **uncertain** (per brief). **uncertain** is the critical third value; binary validators silently approve when in doubt, which is the failure mode we most want to avoid.

### Router (C)

||| |---|---| | Job | Map (validation, ops context) · one of three actions. Explain the decision. || Input | {validation, ops\_context: {customer\_sla\_hours, queue\_depth, doc\_value\_usd, customer\_tier}} || Output | {decision: auto\_approve\|human\_review\|draft\_amendment, rationale (natural language), suggested\_action, discrepancies[]} || Owns | Decision matrix execution, SLA-aware prioritization, rationale generation, amendment-discrepancy enumeration || Does NOT own | Executing the action (downstream services do) |

### Storage + Query (D)

||| |---|---| | Job | Persist every artifact; answer NL questions over them. || Stack | SQLite for the take-home (zero-deps demo); ClickHouse in Nova's production path || Schema | shipments, extractions, validations, decisions, overrides · every agent output is a row, fully self-describing with foreign keys to the shipment || NL·SQL | A small LLM call (Qwen2.5-7B) with the schema in the system prompt + a few-shot of validated SQL examples. Results are executed read-only against the SQLite, and the LLM only generates SELECT. The answer surfaces both the SQL and the result so the operator can verify. || Why this matters | This is the System-of-Outcomes substrate: every "work unit" is stored, queryable, replayable. |

### Minimal UI (E)

||| |---|---| | Job | Show a single shipment's pipeline on one screen · source doc, extracted fields with confidence, validation outcomes, router decision, rationale. || Stack | React (the take-home), shadcn/ui for primitives. In production this becomes a node in Nova's React Flow process editor. || What it shows | Real state from a real run. Not a mockup. |

## 5.4 Agent communication · structured handoff, not shared memory

Not shared memory. Not direct calls. **Typed JSON artifacts, persisted, referenced by ID.**

- Each agent's output is a JSON document, schema-enforced by vLLM `guided_json`, written to the work store before the next stage begins.
- The next agent reads its predecessor's artifact *by ID*, not from in-process memory.
- LangGraph drives the DAG and tracks per-shipment state via its checkpoint (Postgres-backed in production).

Load-bearing properties:

- 1 **Restartability** · any stage can re-run from its inputs.
- 2 **Auditability** · every intermediate artifact is durable evidence.
- 3 **Independence** · each agent can be redeployed, A/B-tested, or migrated without touching the others.

Cost: one extra storage round-trip per stage. Worth it.

## 5.5 State survival mid-crash

- **Per-shipment state machine** in Postgres: `ingested · extracted · validated · routed · completed | escalated | failed`. Transitions atomic.
- **Idempotent stage execution.** Re-running the extractor on the same `doc_id` produces the same artifact path and overwrites cleanly.
- **LangGraph checkpoint.** State persisted after every node; on restart the graph resumes from the last successful node.
- **Lease-based work claiming.** Workers claim a shipment with a 5-minute lease; if a worker dies the lease expires and another picks it up.
- **No agent-to-agent calls.** The orchestrator is the only mover between stages.

At-least-once execution is acceptable because stages are idempotent. Exactly-once is not pursued.

# 6. LLM, Tooling & Orchestration

## 6.1 Per-agent model picks · every choice defended

**Extractor · vision-language model**

- **Primary: Qwen2.5-VL-72B-Instruct** via vLLM. Strong open VLM, structured-output friendly, handles dense documents, supports `guided_json`.
- **Cost tier: Qwen2.5-VL-7B** for digital-native (text-extractable) PDFs. Cuts cost ~10×. The orchestrator's scope-resolution stage classifies the doc as "digital" vs. "scanned" and routes accordingly.
- **Bad-scan fallback: PaddleOCR · text-only Qwen2.5-72B** When the VLM reports unreadable pages, we run classical OCR + text extraction.
- **Why open + self-hosted:** customers are pharma, defense, manufacturing. Their commercial values and HS-coded products are sensitive. Many will not allow document content to leave their VPC. Self-hosted via vLLM is a procurement requirement, not a preference.

**Validator · text reasoning**

- **Primary: Qwen2.5-72B-Instruct.** Strong instruction-following, JSON output, multi-step reasoning.
- **Why 70B not smaller:** validator errors are asymmetrically costly. A wrong "match" is the most expensive failure in the entire system. Spending compute here is correct.

### Router · small text + deterministic rules

- **Rules-first.** A deterministic decision table (YAML) covers ~85% of cases. Only the residual goes to an LLM.
- **LLM tie-breaker + rationale generator: Qwen2.5-7B-Instruct.**
- **Why small:** decision space is tiny (3 actions). A 70B model here is wasted compute.

### Query (D) · NL · SQL

- **Qwen2.5-7B-Instruct** with schema in system prompt + 8 few-shot SQL examples.
- Read-only execution (SELECT only), enforced at the DB user level.
- Returns SQL + result; UI shows both.

## 6.2 Cost / latency / quality summary

| Stage              | Model          | Latency p50 | Cost / call | Quality criticality | Notes                          |
|--------------------|----------------|-------------|-------------|---------------------|--------------------------------|
| Extractor (VL-72B) | Qwen2.5-VL-72B | 6.12s       | ~\$0.04     | High                | errors propagate               |
| Extractor (VL-7B)  | Qwen2.5-VL-7B  | 2.4s        | ~\$0.005    | Low                 | For clean digital PDFs only    |
| Validator          | Qwen2.5-72B    | 2.5s        | ~\$0.02     | Highest             | false-match is the worst error |
| Router (rules)     | n/a            | <50ms       | ~\$0        | Low (deterministic) |                                |
| Router (LLM)       | Qwen2.5-7B     | 0.5.1s      | ~\$0.002    | Low                 | NL · SQL                       |
| Router (NL · SQL)  | Qwen2.5-7B     | 0.5.1s      | ~\$0.002    | Medium              | wrong query · wrong answer     |

**End-to-end target: p50 < 20s, p95 < 45s, cost < \$0.10 per shipment.** Cost dominated by the extractor; caching extractions keyed by doc hash is the single biggest lever.

## 6.3 Vision fallback ladder

Bounded, not infinite:

- 1 VLM primary call with `guided_json`.
- 2 If `unreadable_fields` non-empty · image preprocessing (deskew, denoise, contrast normalize, re-render at higher DPI) · re-call.
- 3 Targeted re-prompt: "Focus on the seal number field; it appears top-right of page 1. Quote the exact characters."
- 4 PaddleOCR + text-LLM fallback.
- 5 Human review with bbox-highlighted page.

Max 2 retries before falling through to human review.

## 6.4 Orchestration framework

**Choice: LangGraph.** Three reasons:

- 1 **It's Nova's actual stack.** GoComet's Nova platform is LangGraph-based. Aligning with their orchestrator is right.
- 2 **Native checkpointing** solves crash recovery without a separate workflow engine.
- 3 **Graph-as-code** lets the same definition run as a take-home prototype and as the production graph behind Nova's React Flow visual editor (YAML under, React Flow on top · both serialize the same graph).

What I do *not* do: put LLM calls inside LangChain abstractions. Each agent is a single function with typed I/O · input contract, vLLM call with `guided_json`, output contract. The LangChain abstractions cost more than they save once contracts are typed.

## 6.5 Where structured output, where not

**Schema-constrained** (`guided_json`): Extractor output, Validator per-rule results, Router decision enum +

structured discrepancies[], tool calls (load ruleset, load schema, query work store).

**Free-form:** rationale strings *inside* structured fields (humans read them); drafted amendment emails (template + free generation, then human-reviewed before send).

Rule of thumb: anything that crosses an agent boundary is schema-constrained. Anything for human consumption can be prose.

## 7. Trust, Hallucination, Evals

### 7.1 Anti-hallucination · five layers

- 1 **Source-bbox required.** Every extracted field must include {page, bbox}. A model that won't point at the source can't claim the value. Non-nullable schema field.
- 2 **Two-pass quote-grounding.** After extraction, second prompt: "For each field, quote the exact text from the document supporting your value." If the quote isn't substring-matchable to the OCR text of the bbox region, the field is marked `hallucination_suspected`.
- 3 **Closed schema.** The JSON schema for a doc type lists exactly the expected fields. `guided_json` rejects invented fields.
- 4 **Explicit unreadable\_fields channel.** Models need a place to put "I don't know." Without it, they hallucinate to fill the slot.
- 5 **Confidence floor.** Fields below 0.7 confidence are never used in auto-approval.

### 7.2 Confidence handling · silent approval is the worst answer

Three bands, three behaviors:

- **0.95** accept.
- **0.7 · 0.95** re-extraction attempted with field-targeted prompt. If still in band, surface to human with bbox.
- **< 0.7** · direct to human review.

For ambiguous extractions (two consignee candidates on the page), both surfaced; human picks. Never randomly choose one. For validation: an `uncertain` rule result forces `human_review` routing regardless of other rules.

### 7.3 Loop, cost, retry controls

- Max retries per stage: 2.
- Per-stage timeouts: Extractor 60s, Validator 20s, Router 5s.
- Per-shipment cost budget: \$0.50 hard cap; exceed · human review.
- Token caps per call: Extractor 4K in / 2K out; Validator 8K in / 1K out; Router 1K in / 256 out.
- **No agent-to-agent calls.** Pipeline is a DAG. Single most effective loop guard.
- Circuit breaker on VLM endpoint: > 10% error rate over 5 min · fail open to OCR fallback.
- Idempotency keys on all downstream side effects.

### 7.4 Evals

**Offline (gated CI):**

- 500 (document, ground-truth extraction) pairs. Adversarial subset: rotated, low-DPI, multi-column, multi-language.
- 200 (extraction + ruleset, ground-truth validation) pairs.
- 100 (validation, ground-truth routing) pairs.
- Metrics: extractor field-F1 (per doc-type and per field), source-bbox IoU, validator per-rule



precision/recall under three-valued labels, router decision accuracy, rationale quality (sampled human rating).

- Any model or prompt change must pass eval before merge.

#### Online (production):

- **Auto-approve precision under 5% sampled human audit.** Of approvals, what fraction does a human agree with? Target · 99%; halt auto-approvals if 97% over a 24h window.
- Why precision and not recall: false approve is the expensive failure. False review is cheap.

## 8. Observability

Explicitly called out because "observability story" is in the rubric. The bar isn't a logging library; it's *can you debug a wrong decision in production by yourself, without re-running the pipeline.*

| Signal | Source | Used by | |---|---|---| | Per-shipment trace | LangSmith / Langfuse (LangGraph native) | Engineer debugging a bad decision | | Per-agent token + cost | vLLM telemetry · ClickHouse | FinOps; per-customer cost dashboards | | Confidence histograms | Extractor output | Drift detection on field quality | | Decision distribution | Router output | Pilot success monitoring; does STP rate drift? | | Override events | UI · work store | Feedback loop into golden eval set | | Stage durations | LangGraph node timings | SLO alerting | | Schema-validation failures | `guided_json` rejection logs | Tells you when the model is fighting the schema (often signals doc-type mismatch upstream) |

Three rules:

- 1 **Every agent call is traced.** Inputs, outputs, model version, latency, token count, cost.
- 2 **Every decision is replayable.** Given the artifacts in the work store, you can re-run any agent stage offline and reproduce its output.
- 3 **Drift is monitored automatically.** A weekly job re-scores last week's auto-approvals against the audit sample; if precision drifts below 97%, the on-call gets paged and auto-approval is paused.

## 9. Cost & Latency Budget

Worked example for 10,000 shipments/month (a credible pilot-to-early-prod volume):

| Component | Cost/shipment | Monthly @ 10k | |---|---|---| | Extractor (60% VL-7B path, 40% VL-72B path) | \$0.018 | \$180 | | Validator (Qwen 72B, single call) | \$0.020 | \$200 | | Router (rules + occasional 7B) | \$0.001 | \$10 | | NL·SQL (sampled by ops, ~3 queries/operator/day) | \$0.001 | (operator-driven, separate budget) | | Storage (ClickHouse @ scale) | \$0.005 | \$50 | | **Total** | **~\$0.044** | **~\$440 / 10k** |

Compared to current human cost: ~15 min of senior ops time per shipment at \$40/hr loaded = \$10/shipment. **Two orders of magnitude headroom.** The unit economics work even if our cost numbers are off by 3×.

Latency:

- p50 end-to-end: · 20s (extraction is the dominant stage)
- p95: · 45s
- Pre-flight check for SU supplier (Liang's flow) needs to be < 60s p95 · same budget.

Two specific levers:

- **Cache extractions by doc hash.** Re-validation under a new rule = pennies + seconds.
- **Doc-type-aware routing.** Digital PDFs · VL-7B (90% latency cut, 87% cost cut on that path). Only scans hit VL-72B.

## 10. Metrics & Pilot Go/No-Go

## 10.1 North-star

**Straight-Through Processing rate with zero downstream defect within 14 days** · the share of shipments that move from doc ingest to action with zero human touch *and* no defect surfaced in the 14-day audit window.

(One number. The "zero defect" tail is what makes it testable; without it the metric is trivially gameable by approving everything.)

## 10.2 Supporting metrics (7)

- 1 **Extractor field-F1** (golden + 5% production audit) · perception quality.
- 2 **Validator three-valued rule-F1** · judgment quality.
- 3 **Auto-approve precision** (production audit) · false-positive rate; the expensive direction.
- 4 **Median time-to-decision** (ingest · router) · system health.
- 5 **Cost per shipment** · unit economics.
- 6 **Exception resolution time** (human-touched docs, ingest · ops sign-off) · human-loop experience.
- 7 **Rule update lead time** (ops request · live in prod) · whether we kept "the engineer needed" promise.

## 10.3 Two-week pilot Go / No-Go

### Pre-pilot Go (all must hold):

- One customer, · 50 shipments expected in 2 weeks.
- Top 3 doc types covered (BoL, Invoice, Packing List); top 10 rules formalized.
- Golden set built and signed off by customer's senior ops.
- Shadow mode week 1 (Nova validates, humans still own decision; we measure agreement).
- InfoSec + compliance sign-off.

### In-flight halts (any one halts auto-approval):

- Auto-approve precision < 97% over any 48h window.
- Any high-severity false-approve on HS code, sanctioned-party, consignee, or declared value.
- Median time-to-decision > 60s for · 1 hour.
- Cost per shipment > \$0.20.
- Customer compliance flags a missing audit-trail element.

### Pilot success exit:

- STP rate · 60%.
- Auto-approve precision · 99%.
- Senior ops reports · 50% time savings in week 2 vs. baseline.

## 11. Demo Plan

End-to-end on real input. Each numbered step maps to one of A·E.

**Setup.** SQLite work-store seeded. One customer ruleset (acme@v1) loaded with 8 rules covering HS-code allow-list, consignee equality, value tolerance, Incoterms allow-list. One sample BoL PDF and one matching commercial invoice on disk.

- 1 **A · Extractor runs on the BoL** Output JSON includes all 8 required fields (consignee, HS code, POL, POD, Incoterms, description, gross weight, invoice number) each with {value, confidence, source: {page, bbox}, quote}. UI shows the source PDF on the left with bboxes overlaid; extracted JSON on the right.
- 2 **B · Validator runs against acme@v1.** Output shows per-rule {rule\_id, status: match|mismatch|uncertain, found, expected, reason}. We deliberately seed

one mismatch (HS code not in allow-list) and one uncertain (value tolerance threshold unset for this customer).

- 3 **C · Router decision** Mismatch on HS code is high-risk · `human_review`. Output includes a written rationale ("HS 8471.30 not in approved list v3; HS codes change duty class so auto-amendment is unsafe") and an enumerated `discrepancies[]` list.
- 4 **D · Storage + Query** All artifacts persisted. Ops asks: "how many shipments were flagged this week?" · NL·SQL agent returns the SQL it generated, the rows it found, and the answer. "3 shipments flagged this week"). A second query: "what's the most common reason for flagging?" · grouped count, with rationale strings.
- 5 **E · UI** One screen shows the whole chain: source doc with bboxes, extracted fields with per-field confidences color-coded, validation table with match/mismatch/uncertain coloring, router decision banner with rationale, override button that writes to the `overrides` table.

**What I'd record for the demo video:** about 90 seconds. (1) drop file · see pipeline run in real time, (2) walk through extracted fields with confidences, (3) walk through validation results with mismatch detail, (4) router decision + rationale, (5) NL query, (6) override demo. Then 30 seconds of "what's under the hood" · show the persisted JSON for one shipment.

## 12. What's Next

If I had two more weeks, in priority order:

- 1 **Cross-doc reconciliation as a first-class step.** Most real rules are inherently cross-doc (BoL · Invoice · Packing-List). v0 treats docs one at a time. First, extend the Validator's context to all docs in a shipment; if rule complexity demands it, introduce a Reconciler agent between Validator and Router. Highest-value next bet.
- 2 **Override · golden-set feedback loop** Every human override becomes a labeled example. Pipe into the eval set and into prompt-time few-shot context for the Validator. This is what makes the system improve over time · the System-of-Outcomes loop made literal.
- 3 **Supplier-facing pre-flight validation.** A portal where Liang drops a draft doc and gets validation feedback in < 60s, before goods ship. Same agents, different UI surface. Compounds the value: defects caught at the supplier, not at the importer.

### Explicitly not next:

- *Fine-tuning a custom extractor.* Premature until we know which fields the open VLMs actually fail on at scale.
- *More doc types speculatively.* Let the next pilot customer's mix dictate this.
- *Self-service rule editor UI.* Until we know whether ops or customer-success owns rules, the UI shape is undefined. YAML in a versioned repo is enough for the first five customers.

## 13. Appendices

### A · Extractor output schema

```
{
  "doc_id": "uuid",
  "doc_type": "bill_of_lading | commercial_invoice | packing_list |
certificate_of_origin",
  "extracted": {
    "consignee_name": { "value": "ACME Corp", "confidence":
0.97, "source": {"page": 1, "bbox": [x,y,w,h]}, "quote": "ACME
Corp" },
```

```

    "hs_code": { "value": "8471.30", "confidence":
0.88, "source": {...}, "quote": "8471.30" },
    "port_of_loading": { "value": "Shenzhen", "confidence":
0.99, "source": {...}, "quote": "Shenzhen, CN" },
    "port_of_discharge": { "value": "Rotterdam", "confidence":
0.99, "source": {...}, "quote": "Rotterdam, NL" },
    "incoterms": { "value": "FOB", "confidence":
0.96, "source": {...}, "quote": "FOB Shenzhen" },
    "description_of_goods": { "value": "Laptop computers, model
X1", "confidence": 0.92, "source": {...}, "quote": "..." },
    "gross_weight_kg": { "value": 1240.5, "confidence":
0.94, "source": {...}, "quote": "1240.5 KG" },
    "invoice_number": { "value":
"INV-2026-441", "confidence": 0.99, "source": {...}, "quote":
"INV-2026-441" }
  },
  "unreadable_fields": [],
  "extractor_version": "qwen2.5-v1-72b@2026-04",
  "extraction_method": "vlm_primary | vlm_retried | ocr_fallback"
}

```

## B · Validator output schema

```

{
  "doc_id": "uuid",
  "rule_set_id": "acme@v1",
  "results": [
    { "rule_id": "hs_code_on_approved_list", "status": "mismatch",
"confidence": 0.95,
    "found": "8471.30", "expected": "one of [8471.41, 8517.62]",
    "reason": "HS 8471.30 not in ACME approved list v3" },
    { "rule_id": "value_within_tolerance", "status": "uncertain",
"confidence": 0.62,
    "found": "Invoice $12,450 / BoL $12,400",
    "expected": "delta · tolerance (unset for this customer)",
    "reason": "Tolerance threshold not configured; cannot
determine match" },
    { "rule_id": "consignee_matches_invoice", "status": "match",
"confidence": 0.99,
    "found": "ACME Corp", "expected": "ACME Corp" }
  ],
  "validator_version": "qwen2.5-72b@2026-04"
}

```

## C · Router output schema

```

{
  "doc_id": "uuid",
  "decision": "human_review",
  "rationale": "HS code 8471.30 is not in ACME's approved list v3.
HS codes determine duty class, so auto-amendment is unsafe · a human
must confirm whether this is a list-update or a supplier error.",
  "discrepancies": [

```

```

    { "field": "hs_code", "found": "8471.30", "expected_one_of":
["8471.41", "8517.62"], "severity": "high" }
  ],
  "suggested_action": { "type": "queue_for_ops", "priority":
"normal", "preload_fields": ["hs_code"] },
  "router_version": "rules@v2 + qwen2.5-7b@2026-04"
}

```

## D · Router decision matrix (v0, customer-configurable)

| Validator state | Router decision | |---|---| | All match, all confidences ≥ 0.9 | auto\_approve | | Any mismatch on a low-risk field (address typo, formatting) | draft\_amendment | | Any mismatch on a high-risk field (HS code, value, consignee, sanctioned party) | human\_review | | Any uncertain rule, or any confidence < 0.9 | human\_review | | Cost budget exceeded or stage timeout | human\_review |

Severity per field is itself customer config (YAML), not code.

## E · Work-store schema (SQLite for the take-home; ClickHouse in prod)

```

CREATE TABLE shipments (
  shipment_id TEXT PRIMARY KEY,
  customer_id TEXT NOT NULL,
  state TEXT NOT NULL,          -- ingested | extracted |
validated | routed | completed | escalated | failed
  created_at TIMESTAMP NOT NULL,
  updated_at TIMESTAMP NOT NULL
);
CREATE TABLE extractions (
  extraction_id TEXT PRIMARY KEY,
  shipment_id TEXT NOT NULL REFERENCES shipments(shipment_id),
  doc_type TEXT NOT NULL,
  extracted_json JSON NOT NULL,  -- full Extractor output
(Appendix A)
  extractor_version TEXT NOT NULL,
  cost_usd REAL NOT NULL,
  latency_ms INTEGER NOT NULL,
  created_at TIMESTAMP NOT NULL
);
CREATE TABLE validations (
  validation_id TEXT PRIMARY KEY,
  shipment_id TEXT NOT NULL REFERENCES shipments(shipment_id),
  extraction_id TEXT NOT NULL REFERENCES
extractions(extraction_id),
  rule_set_id TEXT NOT NULL,
  results_json JSON NOT NULL,   -- full Validator output
(Appendix B)
  validator_version TEXT NOT NULL,
  cost_usd REAL NOT NULL,
  created_at TIMESTAMP NOT NULL
);
CREATE TABLE decisions (
  decision_id TEXT PRIMARY KEY,
  shipment_id TEXT NOT NULL REFERENCES shipments(shipment_id),
  validation_id TEXT NOT NULL REFERENCES

```

```

validations(validation_id),
    decision TEXT NOT NULL,          -- auto_approve | human_review |
draft_amendment
    rationale TEXT NOT NULL,
    discrepancies_json JSON,
    router_version TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL
);
CREATE TABLE overrides (
    override_id TEXT PRIMARY KEY,
    decision_id TEXT NOT NULL REFERENCES decisions(decision_id),
    operator_id TEXT NOT NULL,
    new_decision TEXT NOT NULL,
    rationale TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL
);

```

This is the System-of-Outcomes substrate in concrete form. Every "work unit" is a row; every transition is auditable; the feedback signal from `overrides` is durably captured.

## F · Repo layout (planned)

```

trade-doc-nova/
... README.md                # 1-page: what, how to run, what's
in the demo
... pyproject.toml
... docker-compose.yml      # vllm-serving the two models + the
app
... prd.md                  # this document
... src/
.  ... orchestrator/
.  .  ... graph.py          # LangGraph DAG: scope · context · schema ·
plan+execute · evidence
.  .  ... state.py           # typed state shared across nodes
.  .  ... checkpointer.py    # Postgres-backed checkpointing
.  ... agents/
.  .  ... extractor.py       # vLLM call with guided_json against the
doc-type schema
.  .  ... validator.py       # rule evaluator + LLM reasoning for
uncertain cases
.  .  ... router.py          # rules-first + LLM tie-break +
rationale gen
.  ... store/
.  .  ... schema.sql         # Appendix E
.  .  ... repo.py            # typed repository pattern; one method
per artifact
.  .  ... nl_query.py        # NL·SQL agent (Behaviour D)
.  ... schemas/
.  .  ... doc_types/         # one YAML per doc type · the "generic
engine" config
.  .  ... customers/         # one folder per customer · rules,
severity map, decision matrix
.  ... ui/
.  .  ... App.tsx            # React + shadcn/ui (Behaviour E)

```

```

. . . components/
. . . evals/
. . . golden_extraction/    # 50 docs for the take-home; 500 in
prod
. . . golden_validation/
. . . run_eval.py
... docs/
. . . adr/                  # one ADR per architectural decision
(why LangGraph, why 3 agents, why Qwen, etc.)

```

The ADR folder is deliberate: every defensible tech choice gets one paragraph of "what we chose, what we considered, why we chose what we chose." Reviewable in 15 minutes.

---

## Sources

- Primary VC · *AI agents need systems of work, not record*
- Pragmatic Engineer · *What are Forward Deployed Engineers, and why are they so in demand?*
- Palantir · *A Day in the Life of a Forward Deployed Software Engineer*
- LangChain · *LangGraph: Agent Orchestration Framework*
- Benchmarking Multi-Agent LLM Architectures for Financial Document Processing
- GoComet · corporate site and Nova landing